



Cahier des charges

Application web HomeSkolar

Mise en relation d'élèves en difficulté scolaire
avec des tuteurs bénévoles à distance

Client : HomeSkolar (association)

Prestataire : CodeIguanas — Entité Web

Document : Cahier des charges v1.0

Date : 2026

Sommaire

1. Présentation et contexte du projet

- Le client et son besoin
- Objectifs de l'application
- Les utilisateurs (acteurs)
- Périmètre du projet

2. Spécifications fonctionnelles

- Vue d'ensemble des fonctionnalités
- Gestion des comptes et mise en relation
- Messagerie
- Rencontres et calendrier
- Gestion des tâches
- Traçabilité besoin client → fonctionnalité

3. Veille technologique

- Enjeu et méthode de la veille
- Sources surveillées
- Synthèse et décision

4. Spécifications techniques

- Architecture générale
- Choix technologiques détaillés
- Sécurité et données personnelles
- Réponse aux besoins du client

5. Diagramme de classes (UML)

6. Glossaire

1. Présentation et contexte du projet

1.1 Le client et son besoin

HomeSkolar est une association qui met en relation des enfants en difficulté scolaire avec des bénévoles, à distance. Son ambition est de permettre à *tout* élève, où qu'il soit, d'accéder à un soutien scolaire de qualité.

Concrètement, chaque élève inscrit se voit attribuer un tuteur bénévole. Ce tuteur accompagne l'élève dans son apprentissage au cours de courts rendez-vous hebdomadaires, durant lesquels il l'aide à réaliser ses devoirs et à s'organiser. Pour faciliter ces échanges, HomeSkolar souhaite se doter d'un **nouveau site web** regroupant l'ensemble des outils nécessaires au binôme élève / tuteur.

1.2 Objectifs de l'application

L'application doit répondre à quatre objectifs principaux :

- **Rendre le soutien scolaire accessible à distance**, sans contrainte géographique, à travers une plateforme web unique.
- **Faciliter la communication** entre l'élève et son tuteur entre les rendez-vous.
- **Organiser les rencontres** hebdomadaires grâce à un calendrier partagé.
- **Assurer le suivi du travail** de l'élève au moyen d'une liste de tâches à réaliser d'une rencontre à l'autre.

1.3 Les utilisateurs (acteurs)

Trois profils d'utilisateurs interviennent dans l'application :

Acteur	Rôle
Élève	Enfant inscrit qui bénéficie du soutien scolaire. Il échange avec son tuteur, consulte son calendrier de rencontres et suit la liste des tâches à réaliser.
Tuteur	Bénévole qui accompagne un ou plusieurs élèves. Il planifie les rencontres, communique avec ses élèves et leur assigne des tâches.
Administrateur	Membre de l'association HomeSkolar. Il attribue un tuteur à chaque élève inscrit (constitution des binômes) et gère les comptes en cas de besoin.

L'attribution d'un tuteur à chaque élève étant un besoin explicite du client (« chaque élève inscrit se voit assigner un tuteur bénévole »), un profil **Administrateur** est nécessaire pour constituer ces binômes. Son périmètre reste volontairement limité à la mise en relation et à la gestion des comptes.

1.4 Périmètre du projet

Le présent cahier des charges couvre la **conception du site web** : ses fonctionnalités, ses choix techniques et sa modélisation. Il sert de référence commune au client et à l'équipe de développement (deux développeurs back-end et deux développeurs front-end).

Sont **hors périmètre** de cette première version : l'application mobile native, l'hébergement et le déploiement, ainsi que toute fonctionnalité d'évolution future (proposée à part au responsable et non incluse ci-dessous).

2. Spécifications fonctionnelles

Cette section décrit, dans un langage accessible à tous, **ce que le produit permettra de faire**. Chaque grande fonctionnalité est découpée en sous-fonctionnalités, et l'ensemble est relié aux besoins exprimés par le client (voir la table de traçabilité en fin de section).

2.1 Vue d'ensemble des fonctionnalités

L'application s'organise autour de quatre grandes fonctionnalités :

N°	Fonctionnalité	Description courte
F1	Gestion des comptes et mise en relation	S'inscrire, se connecter, gérer son mot_de_passe et ses données personnelles ; attribuer un tuteur à chaque élève et consulter son binôme.
F2	Messagerie	Échanger des messages, épingler les messages importants, être notifié des messages non lus.
F3	Rencontres et calendrier	Planifier les rendez-vous et les visualiser sur un calendrier.
F4	Gestion des tâches	Assigner et suivre les tâches à réaliser, créer des notes personnelles.

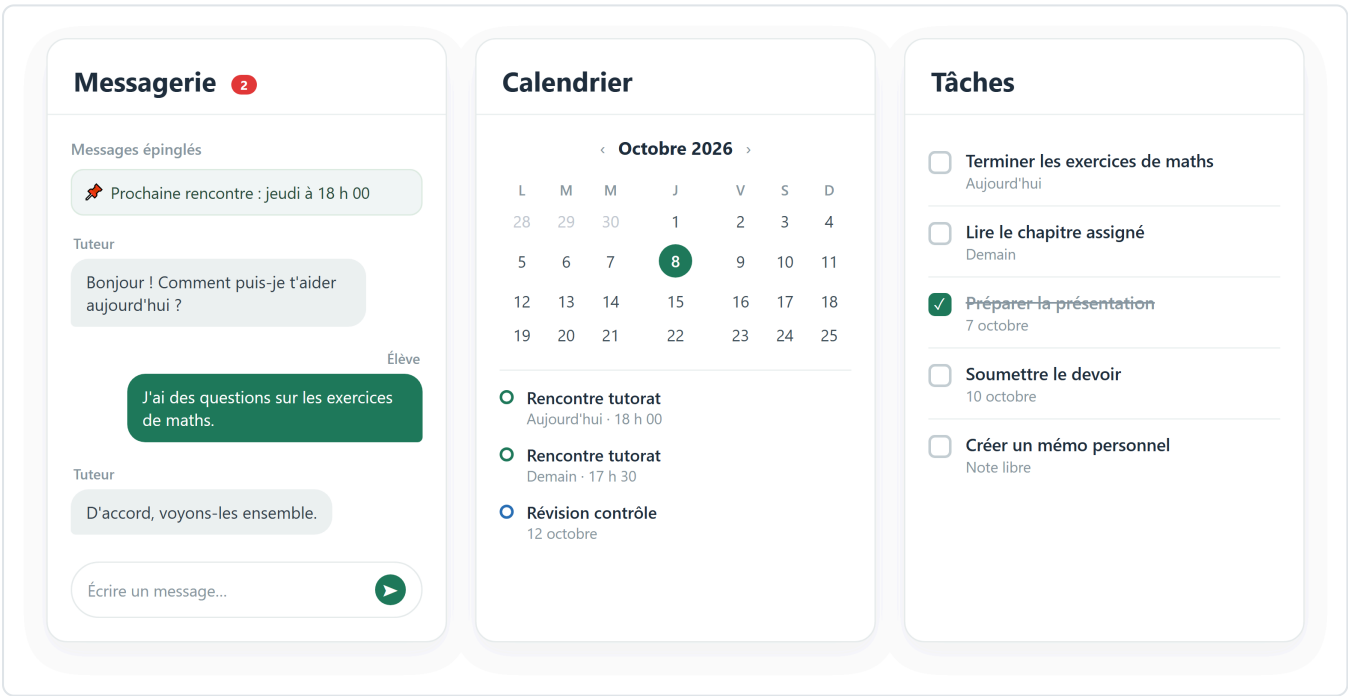


Figure 1 — Maquette d'aperçu de l'espace de travail du binôme : messagerie (F2), calendrier des rencontres (F3) et liste des tâches (F4). Maquette d'illustration, non contractuelle.

2.2 F1 — Gestion des comptes et mise en relation

Cette fonctionnalité permet aux élèves et aux tuteurs de créer et de gérer leur compte en toute autonomie, et à l'association de mettre en relation chaque élève avec un tuteur.

Gestion du compte utilisateur

- **F1.1 — Inscription** : un visiteur crée un compte en tant qu'élève ou tuteur en renseignant ses informations (nom, prénom, adresse email, mot_de_passe, et informations propres à son profil).
- **F1.2 — Connexion** : un utilisateur enregistré accède à son espace personnel à l'aide de son adresse email et de son mot_de_passe.
- **F1.3 — Déconnexion** : un utilisateur ferme sa session de manière sécurisée.
- **F1.4 — Gestion du mot_de_passe** : modification du mot_de_passe depuis l'espace personnel et réinitialisation en cas d'oubli (« mot de passe oublié »).
- **F1.5 — Gestion des données personnelles** : consultation et modification des informations du profil.
- **F1.6 — Suppression du compte** : un utilisateur peut demander la suppression de son compte et de ses données (conformité RGPD).

Mise en relation élève / tuteur

Pour que le soutien puisse commencer, chaque élève doit être associé à un tuteur. Cette mise en relation est réalisée par l'association.

- **F1.7 — Attribution d'un tuteur** : l'administrateur associe un tuteur disponible à un élève inscrit, constituant ainsi un binôme.
- **F1.8 — Consultation du binôme** : l'élève visualise le tuteur qui lui est attribué ; le tuteur visualise la liste des élèves qu'il accompagne.
- **F1.9 — Clôture d'un binôme** : l'administrateur peut mettre fin à un binôme (fin d'accompagnement) sans supprimer l'historique des échanges.

2.3 F2 — Messagerie

La messagerie permet à l'élève et à son tuteur d'échanger simplement entre deux rencontres. Le client souhaite un outil *simple*, sans complexité superflue.

Sous-fonctionnalités

- **F2.1 — Envoi et réception de messages** : au sein d'une conversation, l'élève et le tuteur s'échangent des messages écrits.
- **F2.2 — Épingler un message** : un message important peut être épinglé afin d'être facilement retrouvé par la suite ; il peut aussi être désépinglé.
- **F2.3 — Consultation des messages épinglés** : accès rapide à la liste des messages épinglés de la conversation.
- **F2.4 — Indicateur et notification des messages non lus** : les messages non lus sont signalés, et l'utilisateur est notifié de tout nouveau message.

2.4 F3 — Rencontres et calendrier

Élèves et tuteurs se rencontrent régulièrement et ont besoin de planifier ces rendez-vous. Une page de calendrier classique présente les événements à venir.

Sous-fonctionnalités

- **F3.1 — Planification d'une rencontre** : définir une rencontre en précisant sa date, son heure de début, sa durée et, le cas échéant, un lien de visioconférence.
- **F3.2 — Consultation du calendrier** : visualiser ses rencontres et événements sur un calendrier classique (vues mois / semaine).
- **F3.3 — Modification d'une rencontre** : changer la date, l'heure ou les détails d'un rendez-vous.
- **F3.4 — Annulation d'une rencontre** : annuler un rendez-vous, l'autre membre du binôme en étant informé.

2.5 F4 — Gestion des tâches

À l'issue de chaque rencontre, l'élève dispose d'une liste de tâches à réaliser pour la fois suivante. Chacun (élève ou tuteur) peut également se créer une tâche personnelle en guise de note ou de mémo.

Sous-fonctionnalités

- **F4.1 — Attribution de tâches à l'élève** : le tuteur crée une ou plusieurs tâches pour son élève, avec une échéance (généralement la prochaine rencontre).
- **F4.2 — Création d'une tâche personnelle** : un élève ou un tuteur se crée une tâche pour lui-même (note, mémo).
- **F4.3 — Consultation de la liste des tâches** : chaque utilisateur consulte ses tâches, triées par échéance et par statut.
- **F4.4 — Suivi des tâches** : marquer une tâche comme « en cours » puis « terminée » pour assurer un meilleur suivi.
- **F4.5 — Notification des tâches** : l'élève est notifié dans l'application lorsqu'une nouvelle tâche lui est assignée.

2.6 Traçabilité besoin client → fonctionnalité

Chaque fonctionnalité répond directement à un besoin exprimé dans les notes de réunion du client.

Besoin exprimé par HomeSkolar	Fonctionnalité associée
« Il doit être possible [...] de gérer leur compte utilisateur : l'inscription, la connexion, la gestion de leur mot de passe et de leurs données personnelles. »	F1 — Gestion du compte (F1.1 à F1.6)
« Chaque élève inscrit sur le site se voit assigner un tuteur bénévole. »	F1.7, F1.8 — Mise en relation élève / tuteur
« Nous souhaitons [...] intégrer un système de communication [...] permettre aux élèves d'épingler	F2.1, F2.2, F2.3 — Messagerie

Besoin exprimé par HomeSkolar	Fonctionnalité associée
certaines messages. »	
« L'utilisateur sera bien évidemment notifié pour tout message non lu. »	F2.4 — Notification des messages non lus
« Élèves et tuteurs se voient régulièrement et ont besoin de planifier leurs rencontres [...] une page de calendrier classique. »	F3 — Rencontres et calendrier
« À la fin de chaque rencontre, les élèves vont avoir une liste de tâches à réaliser [...] notifié dans l'application pour un meilleur suivi. »	F4.1, F4.3, F4.4, F4.5 — Tâches assignées et suivi
« Il arrive qu'un élève ou un tuteur décide de se créer une tâche pour lui-même, comme une note ou un mémo. »	F4.2 — Tâche personnelle

3. Veille technologique

3.1 Enjeu et méthode de la veille

La veille technologique consiste à se tenir informé en continu de l'évolution des technologies, de leurs forces et de leurs faiblesses. Son **enjeu** est double : choisir des technologies *pérennes* (maintenues dans le temps, soutenues par une communauté active) et *adaptées* au besoin, afin de limiter les coûts de maintenance et les risques techniques sur la durée de vie de l'application.

Notre méthode privilégie, conformément au principe « préférer le simple au complexe », les solutions les plus simples avant d'augmenter la complexité. Pour chaque technologie envisagée, nous évaluons :

- la **maturité et la stabilité** (versions LTS, ancienneté, fréquence des changements cassants) ;
- la **vitalité de la communauté** (utilisateurs et contributeurs) ;
- la **qualité de la documentation** officielle ;
- l'**adéquation au besoin** du client et aux compétences de l'équipe.

Cette pratique sera poursuivie tout au long du projet : suivi des blogs officiels et des notes de version, lecture d'enquêtes annuelles de référence, abonnement à des newsletters spécialisées (ex. *.NET Blog*, *dev.to*) et veille sur les dépôts GitHub des technologies retenues.

3.2 Sources surveillées

Nous nous appuyons sur des sources variées et complémentaires : documentation officielle de l'éditeur, enquête communautaire et classement indépendant de popularité. Voici trois sources de référence et l'intérêt de chacune.

Source	Type	Intérêt pour notre veille
Microsoft Learn learn.microsoft.com/dotnet	Documentation officielle (éditeur)	Référence faisant autorité sur l'ensemble de l'écosystème retenu (ASP.NET Core, Blazor, Entity Framework Core, SignalR, Identity et SQL Server). Documentation exhaustive, à jour, avec calendrier officiel des versions LTS. Comme notre solution est entièrement Microsoft (full-stack C#), une seule documentation de référence couvre le front-end comme le back-end et la base de données, ce qui garantit cohérence et pérennité.
Stack Overflow Developer Survey survey.stackoverflow.co	Enquête communautaire annuelle	Plus grande enquête mondiale auprès des développeurs. Elle objective la popularité et le niveau d'appréciation des langages, frameworks et bases de données (C#, .NET et SQL Server y figurent parmi les plus utilisés et appréciés), donc la taille de la communauté et la facilité à recruter et à trouver de l'aide.
DB-Engines Ranking db-engines.com/en/ranking	Classement indépendant de popularité (SGBD)	Baromètre de référence mesurant la popularité des systèmes de gestion de bases de données. Microsoft SQL Server y figure durablement parmi les SGBD relationnels les plus utilisés au monde, ce qui conforte notre choix de base de données (maturité, support et abondance de ressources).

Justification de la sélection. Ces sources sont retenues parce qu'elles sont (1) *variées* — documentation d'éditeur, enquête communautaire et classement indépendant —, (2) *fiables et reconnues* dans la profession, et (3) *complémentaires* : la documentation officielle garantit la justesse technique et la pérennité, tandis que l'enquête et le classement mesurent la vitalité réelle et l'adoption des technologies. Le croisement de ces angles fiabilise nos décisions.

3.3 Synthèse et décision

La veille confirme que les technologies de l'écosystème **.NET** (back-end), **Blazor** (front-end en C#) et **SQL Server** (base de données) sont à la fois matures, largement adoptées, bien documentées et portées par des communautés très actives. Elles ne sont pas obsolètes, disposent de versions de support à long terme et répondent simplement aux besoins du projet. Le choix d'un socle entièrement Microsoft permet de développer en un seul langage (C#) du front-end à la base de données. Ces constats fondent les choix détaillés dans les spécifications techniques (section 4).

4. Spécifications techniques

Cette section décrit comment l'application sera construite et comment la technique répond aux besoins du client. Elle est rédigée pour être directement exploitable par l'équipe de développement comme par une future équipe de maintenance.

4.1 Architecture générale

L'application suit une architecture **web client-serveur** à trois niveaux, avec une séparation nette entre le front-end (interface), le back-end (logique métier et API) et la base de données. Le back-end est organisé en **couches** (API → services métier → accès aux données), une structure simple, éprouvée et facile à maintenir.

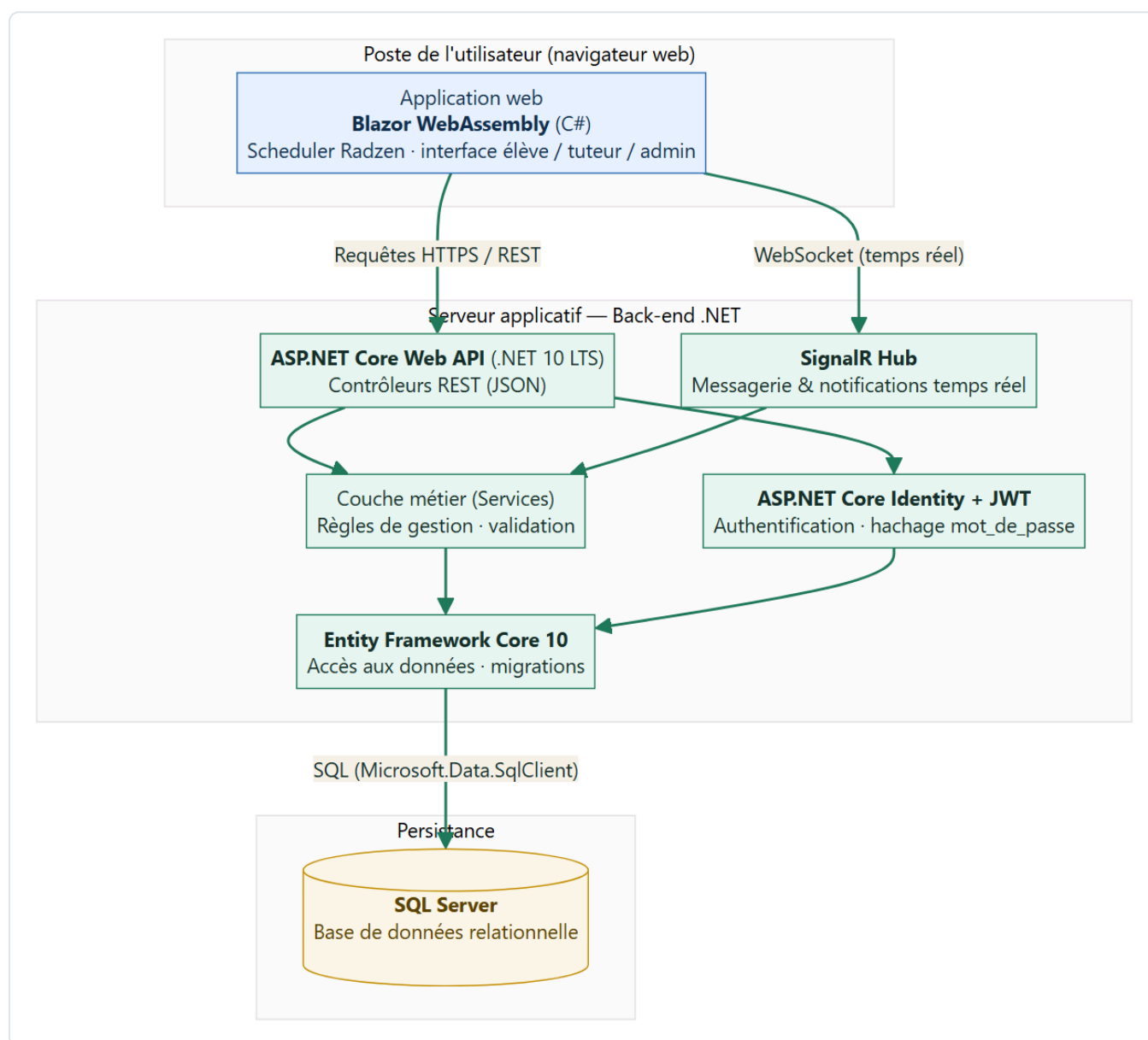


Figure 2 — Architecture technique de l'application HomeSkolar.

Les échanges entre l'interface et le serveur se font via une **API REST** (au format JSON) pour les opérations classiques, complétée par une connexion **temps réel** (WebSocket via SignalR) pour la messagerie et les notifications.

4.2 Choix technologiques détaillés

Pour chaque technologie, nous précisons son rôle et la raison pour laquelle elle est préférée à ses alternatives.

Technologie	Rôle	Pourquoi ce choix	Alternatives écartées
ASP.NET Core (.NET 10 LTS), langage C#	Back-end : API REST, logique métier	Framework back-end mature, performant et multiplateforme, en version à support long terme. Écosystème intégré (Identity, SignalR, EF Core) qui couvre nativement nos besoins. C'est la technologie cœur de l'équipe back-end.	Node.js/Express (typage et structure moins stricts), Django/Python (hors compétences cibles).
Entity Framework Core 10	Accès aux données (ORM)	Permet de manipuler la base via des objets C# et de versionner le schéma grâce aux <i>migrations</i> . Forte productivité et code lisible (LINQ), bien adapté à un modèle relationnel structuré.	Dapper (plus bas niveau, moins productif pour du CRUD), ADO.NET brut (verbeux).
Microsoft SQL Server	Base de données relationnelle	SGBD relationnel de Microsoft, robuste et largement utilisé en entreprise. Le modèle relationnel correspond parfaitement à des données fortement liées (utilisateurs, binômes, messages, rencontres, tâches). Intégration native avec l'écosystème .NET / EF Core et une édition gratuite (Express / Developer) disponible pour le développement.	PostgreSQL (excellent SGBD open source, mais hors de l'écosystème Microsoft retenu), bases NoSQL (inadaptées à des données relationnelles).
ASP.NET Core Identity + JWT	Authentification et sécurité	Solution officielle de gestion des comptes (inscription, connexion, mot_de_passe haché). Les jetons JWT sécurisent les échanges entre l'interface Blazor et l'API sans stocker d'état côté serveur.	Développer son propre système d'authentification (risqué et coûteux à sécuriser).
SignalR	Messagerie et notifications en temps réel	Brique native d'ASP.NET Core pour le temps réel. Elle gère automatiquement les WebSockets et répond simplement au besoin «	Interrogation périodique du serveur (« polling ») : plus lourd et moins réactif ; bibliothèques

Technologie	Rôle	Pourquoi ce choix	Alternatives écartées
		messaging sans complexité » et « notification des messages non lus ».	tierces (dépendance supplémentaire).
Blazor WebAssembly (C#)	Front-end : interface utilisateur	Framework d'interface web officiel de Microsoft permettant d'écrire le front-end en C# , le même langage que le back-end. L'équipe travaille ainsi dans un seul écosystème (compétences mutualisées, code partagé, typage fort de bout en bout), avec une intégration native à .NET et SignalR.	React, Angular, Vue (frameworks JavaScript performants, mais imposant un second langage et un écosystème distinct du back-end .NET).
Radzen Blazor (composant Scheduler)	Affichage du calendrier	Bibliothèque de composants Blazor open source offrant un composant calendrier/agenda prêt à l'emploi, avec les vues mois/semaine attendues pour une « page de calendrier classique » sans la redévelopper.	Développement d'un calendrier sur mesure (effort élevé, peu pertinent).
Swagger / OpenAPI	Documentation de l'API	Génère automatiquement la documentation interactive de l'API REST, facilitant le travail commun des équipes front-end et back-end et la maintenance.	Documentation rédigée à la main (vite obsolète).

4.3 Sécurité et données personnelles

L'application manipule des données de mineurs ; la sécurité est donc primordiale :

- les **mot_de_passe** ne sont jamais stockés en clair : ils sont hachés (PBKDF2 via ASP.NET Core Identity) ;
- les échanges transitent en **HTTPS** et l'accès à l'API est protégé par jeton **JWT** ;
- chaque utilisateur n'accède qu'à **ses propres données** et à celles de son binôme (contrôle des autorisations par rôle) ;
- conformément au **RGPD**, l'utilisateur peut consulter, modifier et supprimer ses données personnelles.

4.4 Réponse aux besoins du client

Besoin	Réponse technique
Comptes (inscription, connexion, mot_de_passe, données personnelles)	ASP.NET Core Identity + JWT ; profils persistés via EF Core dans SQL Server.

Besoin	Réponse technique
Messagerie simple, épinglage, notification des non-lus	API REST pour l'historique et l'épinglage ; SignalR pour la réception et la notification en temps réel.
Calendrier classique des rencontres	Rencontres gérées par l'API ; affichage via le composant Scheduler de Radzen Blazor (vues mois/semaine).
Tâches à réaliser et notes personnelles, avec suivi	Entité Tache (assignée ou personnelle) gérée par l'API ; notification via SignalR.
Mise en relation élève / tuteur	Entité Binome créée par l'administrateur, contrôle d'accès par rôle.

5. Diagramme de classes (UML)

Le diagramme ci-dessous modélise les objets de l'application et leurs relations, selon la nomenclature UML. Il est directement déduit des spécifications fonctionnelles et conçu pour être **extensible** (ajout futur de nouvelles classes sans remise en cause de l'existant).

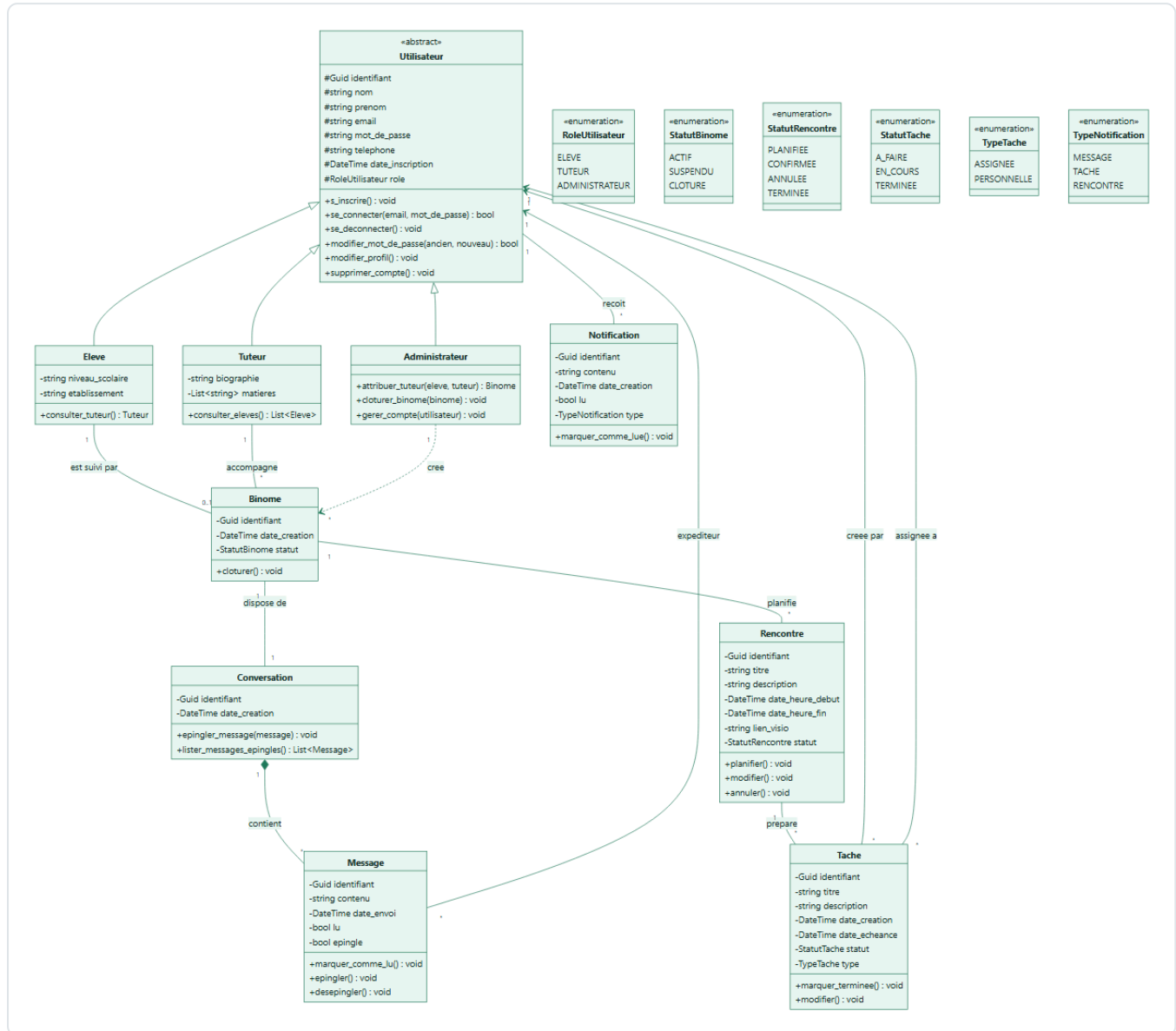


Figure 3 — Diagramme de classes de l'application HomeSkolar.

5.1 Description des classes

Classe	Description
Utilisateur (abstraite)	Classe parente regroupant les données et actions communes à tout compte : identité, email, mot_de_passe, et opérations d'inscription/connexion/gestion du profil.
Élève	Spécialise Utilisateur . Ajoute le niveau scolaire et l'établissement ; consulte son tuteur.

Classe	Description
Tuteur	Spécialise <code>Utilisateur</code> . Ajoute une biographie et les matières enseignées ; consulte ses élèves.
Administrateur	Spécialise <code>Utilisateur</code> . Attribue les tuteurs aux élèves et gère les comptes.
Binôme	Relation entre un élève et un tuteur (date de création, statut). Porte la conversation, les rencontres et sert de cadre au suivi.
Conversation	Espace d'échange d'un binôme ; contient les messages et gère l'épinglage.
Message	Message d'une conversation : contenu, date, statut lu/non lu, épinglé ou non, avec son expéditeur.
Rencontre	Rendez-vous planifié d'un binôme : titre, dates de début/fin, lien de visio, statut. Affiché dans le calendrier.
Tâche	Travail à réaliser : titre, description, échéance, statut et type (assignée par le tuteur ou personnelle). Peut préparer une rencontre.
Notification	Information adressée à un utilisateur (nouveau message, nouvelle tâche, rencontre), avec statut lu/non lu.

5.2 Principales relations

- **Héritage** : `Élève`, `Tuteur` et `Administrateur` héritent d' `Utilisateur`.
- **Binôme** : un tuteur accompagne plusieurs binômes ; un élève appartient au plus à un binôme actif. L'administrateur crée les binômes.
- **Messagerie** : un binôme dispose d'une conversation, qui contient (composition) plusieurs messages, chacun ayant un expéditeur.
- **Rencontres** : un binôme planifie plusieurs rencontres ; une rencontre peut préparer plusieurs tâches.
- **Tâches** : chaque tâche a un créateur et un destinataire (utilisateurs).
- **Notifications** : un utilisateur reçoit plusieurs notifications.

Les énumérations (`RoleUtilisateur`, `StatutBinome`, `StatutRencontre`, `StatutTache`, `TypeTache`, `TypeNotification`) garantissent des valeurs cohérentes et facilitent l'évolution.

6. Glossaire

Terme	Définition
API REST	Interface permettant à l'application web de communiquer avec le serveur via des requêtes standardisées (au format JSON).
Binôme	Couple formé d'un élève et de son tuteur attribué.
JWT	<i>JSON Web Token</i> : jeton sécurisé prouvant l'identité de l'utilisateur à chaque requête.
LTS	<i>Long Term Support</i> : version d'une technologie bénéficiant d'un support prolongé, gage de stabilité.
ORM	<i>Object-Relational Mapping</i> : outil reliant les objets du code à la base de données.
RGPD	Règlement général sur la protection des données : cadre légal européen sur les données personnelles.
SignalR	Technologie .NET permettant des échanges en temps réel (messagerie, notifications).
UML	<i>Unified Modeling Language</i> : langage standard de modélisation logicielle.
WebSocket	Canal de communication permanent entre le navigateur et le serveur, pour le temps réel.